# Bölüm 15: Pandas

SE 422 · Introduction to Data Science · Konu özeti

Kaynak slayt destesi: Pandas
7 Kasım 2026

# İçindekiler

**Bölüm 15 · Pandas . . . . . 3**

Bölümün amacı . . . . . 3

Konu anlatımı . . . . . 3

Pandas nedir, neden pandas? (Slayt 1–5) . . . . . 3

Series (Slayt 6–8) . . . . . 4

DataFrame ve veri hizalama (Slayt 9–12) . . . . . 4

Veriyi incelemek (Slayt 13–17) . . . . . 5

I/O: okuma ve yazma (Slayt 19–23) . . . . . 6

Seçim: [], loc, iloc (Slayt 25–31) . . . . . 7

Boolean indeksleme (Slayt 32–34) . . . . . 8

Eksik veri (Slayt 36–44) . . . . . 8

Birleştirme: concat ve merge (Slayt 45–50) . . . . . 9

groupby: böl, uygula, birleştir (Slayt 51–58) . . . . . 10

Yeniden şekillendirme: melt ve pivot_table (Slayt 59–62) . . . . . 11

Zaman serileri (Slayt 63–66) . . . . . 12

Özet ve kaynaklar (Slayt 67–68) . . . . . 13

Sık yapılan hatalar . . . . . 13

Kısa tekrar soruları . . . . . 14

Cevaplar . . . . . 14

# Bölüm 15 · Pandas

## Bölümün amacı

Bu bölüm tablo biçimindeki veriyle çalışmanın standart aracı **pandas**'ı, tipik bir veri analizi akışını izleyerek öğretir. Sonunda şunları yapabilmelisin:

- **Series** ve **DataFrame** oluşturmak; pandas'ın **etiketlere göre veri hizalama** ilkesini açıklamak
- Veriyi incelemek (`shape`, `dtypes`, `info`, `head`, `describe`), CSV okumak ve yazmak
- `[]`, `loc`, `iloc` ve boolean indeksleme ile veri seçmek ve filtrelemek
- Eksik veriyi bulmak, silmek ve doldurmak
- `concat` ve `merge` ile tabloları birleştirmek; `groupby` ile böl-uygula-birleştir analizi yapmak
- `melt` ve `pivot_table` ile yeniden şekillendirmek; tarihlerle ve zaman serileriyle çalışmak

## Konu anlatımı

### Pandas nedir, neden pandas? (Slayt 1–5)

**Pandas**, Python için açık kaynaklı, yüksek performanslı bir veri işleme ve analiz kütüphanesidir; adı ekonometrideki **"panel data"** teriminden gelir. İki temel yapısı vardır: tek boyutlu, etiketli **Series** ve iki boyutlu, etiketli, sütunları farklı tiplerde olabilen **DataFrame**.

- **Elektronik tablolara göre:** büyük veride (milyonlarca satır) yavaşlamaz; işlemler betiklere yazıldığı için analiz **tekrarlanabilir, test edilebilir ve otomatik** olur.
- **SQL'e göre:** SQL veriyi veritabanından çekmek içindir; pandas çekilen veriyle yapılan **analiz ve modelleme** içindir. `GROUP BY`, `JOIN` gibi işlemlerin karşılıkları vardır.
- **R'a göre:** R'ın `data.frame` kolaylığını Python'a getirir.

Kurulum `pip install pandas`; kural olarak `import pandas as pd` ile kullanılır.

PYTHON
```python
import pandas as pd
print(type(pd.__version__).__name__, pd.__version__.split('.')[0])
```

ÇIKTI
```
str 2
```

## Series (Slayt 6–8)

**Series**, tek boyutlu ve **etiketli** bir dizidir; etiketlerine **index** denir. Listeden (index 0, 1, 2…), sözlükten (anahtarlar index olur) ya da `index=` ile oluşturulur.

PYTHON

```python
import pandas as pd
s1 = pd.Series([10, 20, 30])
print(s1)
s2 = pd.Series({'Alice': 85, 'Bob': 92})
print(s2['Bob'])
s3 = pd.Series([1.5, 2.5, 3.5], index=['a', 'b', 'c'], name='olcum')
print(s3.index.tolist(), s3.values, s3.dtype, s3.name, s3.size, s3.shape)
```

ÇIKTI

```
0    10
1    20
2    30
dtype: int64
92
['a', 'b', 'c'] [1.5 2.5 3.5] float64 olcum 3 (3,)
```

## DataFrame ve veri hizalama (Slayt 9–12)

**DataFrame**, aynı index'i paylaşan Series'lerden oluşan bir sözlük gibidir. En yaygın oluşturma yolu **listelerden oluşan sözlüktür**: anahtarlar sütun adları olur ve **tüm listeler aynı uzunlukta** olmalıdır.

PYTHON

```python
import pandas as pd
df = pd.DataFrame({'Name': ['Alice', 'Bob', 'Charlie', 'David'],
                   'Age': [24, 27, 22, 32],
                   'Major': ['CS', 'Math', 'Physics', 'CS']})
print(df)
```

ÇIKTI

```
      Name  Age    Major
0    Alice   24       CS
1      Bob   27     Math
2  Charlie   22  Physics
3    David   32       CS
```

Series'lerden oluşan sözlükte index, tüm index'lerin **birleşimidir**; bir Series'te olmayan etiket **NaN** olur. Bu, pandas'ın temel ilkesi **içsel veri hizalamanın** bir sonucudur: işlemler konuma göre değil **etiketlere göre** eşleşir, eşi olmayan etiket NaN olur. Excel gibi konuma dayalı araçlarda sık görülen yanlış

hesaplamaları önler.

PYTHON
```python
import pandas as pd
d = {'one': pd.Series([1.0, 2.0, 3.0], index=['a', 'b', 'c']),
     'two': pd.Series([1.0, 2.0, 3.0, 4.0], index=['a', 'b', 'c', 'd'])}
print(pd.DataFrame(d))
a = pd.Series([1, 2, 3], index=['a', 'b', 'c'])
b = pd.Series([10, 20, 30], index=['b', 'c', 'd'])
print((a + b).tolist())
```

ÇIKTI
```
   one  two
a  1.0  1.0
b  2.0  2.0
c  3.0  3.0
d  NaN  4.0
[nan, 12.0, 23.0, nan]
```

## Veriyi incelemek (Slayt 13–17)

- `df.shape` → (satır, sütun) · `df.dtypes` → sütun tipleri (**öznitelik**, parantez yok)
- `df.info()` → özet **yazdırır** (metot): satır sayısı, sütun başına **Non-Null Count** (eksik veri işareti) ve tip. String sütunlar `object` görünür; sayısal olması gereken bir sütunun `object` olması temizlik gerektiğini gösterir.
- `df.head(n)` / `df.tail(n)` → ilk/son n satır (varsayılan 5)
- `df.describe()` → **sayısal** sütunlar için count, mean, std, min, %25, %50 (medyan), %75, max

**Merkezi eğilim** (mean, medyan, mod) tipik değeri, **yayılım** (std, varyans, aralık) dağınıklığı gösterir. Q1 (%25) ve Q3 (%75) çeyreklerdir; **Q3 − Q1 = IQR**.

PYTHON
```python
import pandas as pd
df = pd.DataFrame({'Name': ['Alice', 'Bob', 'Charlie', 'David'],
                   'Age': [24, 27, 22, 32],
                   'Major': ['CS', 'Math', 'Physics', 'CS']})
print(df.shape)
print(df.dtypes)
df.info()
print(df.describe())
q1, q3 = df['Age'].quantile(0.25), df['Age'].quantile(0.75)
print('IQR:', q3 - q1)
```

ÇIKTI

```
(4, 3)
Name     object
Age       int64
Major    object
dtype: object
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4 entries, 0 to 3
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Name    4 non-null      object
 1   Age     4 non-null      int64
 2   Major   4 non-null      object
dtypes: int64(1), object(2)
memory usage: 224.0+ bytes
             Age
count   4.000000
mean   26.250000
std     4.349329
min    22.000000
25%    23.500000
50%    25.500000
75%    28.250000
max    32.000000
IQR: 4.75
```

## I/O: okuma ve yazma (Slayt 19–23)

Okuyucular `pd.read_*` (csv, json, html, excel, parquet, sql), yazıcılar `df.to_*` biçimindedir. `pd.read_csv` yol, URL ya da `.read()` metodu olan dosya benzeri bir nesne alır. Parametreleri veriyi **okurken temizler**: `sep`, `header`, `index_col`, `usecols` (bellek tasarrufu), `parse_dates`, `na_values`. `pd.read_excel` **openpyxl** gerektirir; `sheet_name=None` tüm sayfaları sözlük olarak okur.

PYTHON

```python
import io
import pandas as pd
metin = '''PassengerId,Survived,Age,Date
1,0,22,2024-01-05
2,1,?,2024-01-06
3,1,26,2024-01-07'''
df = pd.read_csv(io.StringIO(metin), index_col='PassengerId',
                 usecols=['PassengerId', 'Survived', 'Age', 'Date'],
                 na_values=['?'], parse_dates=['Date'])
print(df)
print(df.dtypes)
```

ÇIKTI

```
             Survived   Age       Date
PassengerId
1                   0  22.0 2024-01-05
2                   1   NaN 2024-01-06
3                   1  26.0 2024-01-07
Survived             int64
Age                float64
Date        datetime64[ns]
dtype: object
```

`to_csv` varsayılan olarak index'i de yazar; dosya geri okununca bu, **"Unnamed: 0"** sütunu olarak döner. Bu yüzden CSV'ye yazarken neredeyse her zaman `index=False` kullanılır.

PYTHON

```python
import pandas as pd
df = pd.DataFrame({'Name': ['Ada', 'Can'], 'Age': [30, 25]})
df.to_csv('veri.csv')
print(pd.read_csv('veri.csv').columns.tolist())
df.to_csv('veri.csv', index=False)
print(pd.read_csv('veri.csv').columns.tolist())
```

ÇIKTI

```
['Unnamed: 0', 'Name', 'Age']
['Name', 'Age']
```

## Seçim: [], loc, iloc (Slayt 25–31)

Etiketler üç amaca hizmet eder: **tanımlama**, **hizalama**, **seçim**. `df['Age']` bir **Series**, `df[['Age', 'Name']]` bir **DataFrame** döndürür. Açık ve güvenli kod için:

| | `loc` | `iloc` |
|---|---|---|
| Temel | **etiket** | **tam sayı konum** |
| Dilimin bitişi | **dahil** | **hariç** |
| Ne zaman? | string/datetime index, etiketle çalışırken | konumla çalışırken |

PYTHON

```python
import pandas as pd
df = pd.DataFrame({'Age': [24, 27, 22], 'Major': ['CS', 'Math', 'Physics']},
                  index=['Alice', 'Bob', 'Charlie'])
print(type(df['Age']).__name__, type(df[['Age']]).__name__)
print(df.loc['Bob', 'Age'], df.iloc[1, 0])
print(df.loc['Alice':'Bob', 'Age'].tolist())
print(df.iloc[0:2, 0].tolist())
x = pd.DataFrame({'x': [10, 20, 30, 40, 50]})
```

PYTHON (DEVAM)

```
print(len(x.loc[1:3]), len(x.iloc[1:3]))
```

ÇIKTI

```
Series DataFrame
27 27
[24, 27]
[24, 27]
3 2
```

## Boolean indeksleme (Slayt 32–34)

Filtreleme, if-then'in **vektörleştirilmiş** karşılığıdır: önce koşuldan bir boolean **maske** oluşur, sonra `df[maske]` ya da `df.loc[maske, sütun]` ile uygulanır. Koşullar `&` (ve), `|` (veya), `~` (değil) ile birleştirilir ve **her biri parantez içine** alınır. Filtrelenen satırlar orijinal index etiketlerini korur.

PYTHON

```
import pandas as pd
df = pd.DataFrame({'Age': [22, 38, 26, 35, 54, 2],
                   'Sex': ['male', 'female', 'female', 'female', 'male',
↪ 'male'],
                   'Pclass': [3, 1, 3, 1, 1, 3]})
mask = df['Age'] > 30
print(mask.tolist())
print(df[mask])
print(df[(df['Age'] > 30) & (df['Sex'] == 'male')].index.tolist())
print(len(df[(df['Pclass'] == 1) | (df['Age'] < 5)]), len(df[~(df['Pclass'] ==
↪ 3)]))
```

ÇIKTI

```
[False, True, False, True, True, False]
   Age     Sex  Pclass
1   38  female       1
3   35  female       1
4   54    male       1
[4]
4 3
```

## Eksik veri (Slayt 36–44)

Eksik değerler tipe göre gösterilir: `np.nan` (float), `pd.NaT` (tarih/saat), `None` (object). NaN bir float olduğu için tam sayı sütununa tek bir NaN girince sütun **float64'e yükseltilir**; **nullable** tipler (`'Int64'`, `'boolean'`, `'string'`) bunu `pd.NA` ile önler.

`isna()` (eşi `isnull()`) True/False tablosu verir; `.sum()` ile **sütun başına eksik sayısı** bulunur.

PYTHON
```
import pandas as pd
print(pd.Series([1, 2, None]).dtype, pd.Series([1, 2, None],
↪ dtype='Int64').dtype)
df = pd.DataFrame({'A': [1, None, 3, None], 'B': [4, None, None, 8], 'C': [7,
↪ None, 9, 10]})
print(df.isna().sum().tolist())
```

ÇIKTI
```
float64 Int64
[2, 2, 1]
```

`dropna()` eksik içeren satırları siler: `axis=1` sütunları, `how='all'` yalnızca tamamı eksik olanları, `thresh=n` en az n geçerli değeri olanları **tutar**. `fillna()` doldurur: sabit değerle, hesaplanmış değerle (ör. ortalama) ya da sıralı olarak (ileri/geri doldurma). Kategorik sütunlar genellikle **mod** (en sık değer) ile doldurulur.

PYTHON
```
import pandas as pd
df = pd.DataFrame({'A': [1, None, 3, None], 'B': [4, None, None, 8], 'C': [7,
↪ None, 9, 10]})
print(df.dropna().shape, df.dropna(how='all').shape, df.dropna(axis=1,
↪ thresh=3).columns.tolist())
s = pd.Series([1.0, None, None, 4.0])
print(s.fillna(0).tolist(), s.ffill().tolist(), s.bfill().tolist(), s.fillna(
↪ s.mean()).tolist())
e = pd.Series(['S', 'C', None, 'S'])
print(e.fillna(e.mode()[0]).tolist())
```

ÇIKTI
```
(1, 3) (3, 3) ['C']
[1.0, 0.0, 0.0, 4.0] [1.0, 1.0, 1.0, 4.0] [1.0, 4.0, 4.0, 4.0] [1.0, 2.5, 2.5,
↪ 4.0]
['S', 'C', 'S', 'S']
```

**Ek bilgi (resmi dokümantasyon):** pandas 2.1'den itibaren `fillna(method='ffill')` kullanımdan kaldırılma uyarısı verir; yerine `ffill()` ve `bfill()` metotları önerilir. Kaynak: pandas.pydata.org/docs (pandas.DataFrame.fillna).

## Birleştirme: concat ve merge (Slayt 45–50)

- `pd.concat`: aynı yapıdaki tabloları **yığar** (ör. aylık raporlar). Varsayılan `axis=0` alt alta ekler ve orijinal index'leri korur; `ignore_index=True` temiz bir 0, 1, 2… index'i oluşturur. `axis=1` yan yana koyar ve index'e göre hizalar.
- `pd.merge`: aynı varlıklar hakkındaki farklı bilgileri bir **anahtarla** bağlar (SQL JOIN). `on` (ortak sütun), `left_on`/`right_on` (farklı adlı anahtar),

`left_index`/`right_index`. `how`: **inner** (varsayılan, kesişim), **outer** (birleşim), **left**, **right**. `validate='1:1'`, `'1:m'`, `'m:1'` çıktıyı değiştirmez; anahtarların benzersizliğini **doğrular**.

PYTHON

```
import pandas as pd
jan = pd.DataFrame({'urun': ['A', 'B'], 'satis': [10, 20]})
feb = pd.DataFrame({'urun': ['A', 'C'], 'satis': [15, 5]})
print(pd.concat([jan, feb]).index.tolist(), pd.concat([jan, feb],
↪ ignore_index=True).index.tolist())
df1 = pd.DataFrame({'student_id': ['s1', 's2', 's3'], 'name': ['Ada', 'Can',
↪ 'Ece']})
df2 = pd.DataFrame({'student_id': ['s1', 's2', 's4'], 'course': ['Math',
↪ 'Econ', 'Physics']})
for how in ['inner', 'outer', 'left', 'right']:
    print(how, len(pd.merge(df1, df2, on='student_id', how=how)))
print(pd.merge(df1, df2, on='student_id', how='left'))
```

ÇIKTI

```
[0, 1, 0, 1] [0, 1, 2, 3]
inner 2
outer 4
left 3
right 3
  student_id name course
0         s1  Ada   Math
1         s2  Can   Econ
2         s3  Ece    NaN
```

PYTHON

```
import pandas as pd
a = pd.DataFrame({'k': [1, 1], 'v': [5, 6]})
b = pd.DataFrame({'k': [1], 'w': [7]})
pd.merge(a, b, on='k', validate='one_to_one')
```

ÇIKTI

```
pandas.errors.MergeError: Merge keys are not unique in left dataset; not a
↪ one-to-one merge
```

## groupby: böl, uygula, birleştir (Slayt 51–58)

1. **Böl:** `df.groupby('Sex')` veriyi gruplara ayırır; bu çağrı **henüz hesaplama yapmaz**, tembel bir plan (DataFrameGroupBy) döndürür.
2. **Uygula:** her gruba bir fonksiyon uygulanır: **toplama** (`mean`, `sum`, `count`, `agg`), **dönüştürme** (`transform`: aynı şekil, grup değeri her satıra yayılır), **filtreleme** (`filter`: koşulu sağlamayan grupları tamamen atar).

3 **Birleştir:** toplama sonuçları **grup anahtarlarıyla** indekslenir; dönüştürme ve filtreleme sonuçları **orijinal index'i** korur.

PYTHON
```python
import pandas as pd
df = pd.DataFrame({'Pclass': [3, 1, 3, 1, 3, 3, 1, 3],
                   'Sex': ['male', 'female', 'female', 'female', 'male',
↪ 'male', 'male', 'male'],
                   'Age': [22, 38, 26, 35, 35, None, 54, 2],
                   'Survived': [0, 1, 1, 1, 0, 0, 0, 0]})
print(df.groupby('Sex')['Age'].mean().round(2).to_dict())
print(df.groupby('Pclass')['Survived'].mean().round(2).to_dict())
ozet = df.groupby('Pclass').agg(avg_age=('Age', 'mean'), num_survived=(
↪ 'Survived', 'sum'))
print(ozet)
df['Age'] = df['Age'].fillna(df.groupby('Pclass')['Age'].transform('mean'))
print(df['Age'].round(1).tolist())
print(len(df.groupby('Pclass').filter(lambda g: len(g) >= 4)))
```

ÇIKTI
```
{'female': 33.0, 'male': 28.25}
{1: 0.67, 3: 0.2}
          avg_age  num_survived
Pclass
1       42.333333             2
3       21.250000             1
[22.0, 38.0, 26.0, 35.0, 35.0, 21.2, 54.0, 2.0]
5
```

## Yeniden şekillendirme: melt ve pivot_table (Slayt 59–62)

**Tidy** veride her değişken bir sütun, her gözlem bir satırdır. `pd.melt` sütun başlıkları değer olan (ör. yıllar) **geniş** tabloyu **uzun** formata çevirir (`id_vars`, `var_name`, `value_name`). `pivot_table` tersini yapar: `index` satırlara, `columns` sütunlara, `values` hücrelere gider; aynı hücreye birden çok değer düşerse `aggfunc` uygulanır (varsayılan **ortalama**). pivot_table, groupby + `unstack` birleşimidir.

PYTHON
```python
import pandas as pd
genis = pd.DataFrame({'ulke': ['TR', 'DE'], '2022': [85, 83], '2023': [86,
↪ 84]})
uzun = pd.melt(genis, id_vars=['ulke'], var_name='yil', value_name='nufus')
print(uzun)
df = pd.DataFrame({'Sex': ['m', 'f', 'm', 'f', 'm'], 'Pclass': [1, 1, 3, 3, 3],
                   'Survived': [1, 1, 0, 1, 0], 'Age': [40, 30, 20, 25, 30]})
print(df.pivot_table(values='Age', index='Sex', columns='Pclass'))
print(df.pivot_table(values='Survived', index='Sex', columns='Pclass',
↪ aggfunc='sum'))
```

ÇIKTI

```
  ulke   yil  nufus
0   TR  2022     85
1   DE  2022     83
2   TR  2023     86
3   DE  2023     84
Pclass     1     3
Sex
f       30.0  25.0
m       40.0  25.0
Pclass  1  3
Sex
f       1  1
m       1  0
```

## Zaman serileri (Slayt 63–66)

Pandas finansal analiz için geliştirildiğinden tarihlerle çok güçlüdür. Tipler: **Timestamp** (tek an), **DatetimeIndex** (Timestamp'lerden oluşan index), **Timedelta** (süre), **Period** (ay, çeyrek gibi bir aralık). Çoğu tarih string (`object`) olarak yüklenir; `pd.to_datetime` onları Timestamp'e çevirir. Sonra `.dt` ile parçalarına erişilir, tarih aritmetiği yapılır.

PYTHON

```python
import pandas as pd
t = pd.Timestamp('2024-03-15 10:30')
print(t.year, t.month, pd.Timestamp('2024-03-20') - t, pd.Period('2024-03',
↪ freq='M'))
df = pd.DataFrame({'tarih': ['2024-01-05', '2024-02-10', '2024-02-29'],
↪ 'satis': [100, 150, 120]})
print(df['tarih'].dtype, end=' → ')
df['tarih'] = pd.to_datetime(df['tarih'])
print(df['tarih'].dtype)
print(df['tarih'].dt.month.tolist(), df['tarih'].dt.day_name().tolist())
print((df['tarih'].max() - df['tarih'].min()).days)
```

ÇIKTI

```
2024 3 4 days 13:30:00 2024-03
object → datetime64[ns]
[1, 2, 2] ['Friday', 'Saturday', 'Thursday']
55
```

Tarihler index olunca (**DatetimeIndex**) satırlar tarih string'leriyle seçilir (`loc` dilimi bitişi dahil eder). `pd.date_range` düzenli tarihler üretir. `resample` örnekleme sıklığını değiştirir: groupby gibi zamanı aralıklara bölüp her aralığa bir toplama uygular (ör. günlük → haftalık, aylık).

PYTHON

```python
import pandas as pd
idx = pd.date_range('2024-01-01', periods=14, freq='D')
ts = pd.Series(range(1, 15), index=idx)
print(ts.loc['2024-01-03':'2024-01-05'].tolist())
print(ts.resample('W').sum())
print(ts.resample('MS').mean())
```

ÇIKTI

```
[3, 4, 5]
2024-01-07    28
2024-01-14    77
Freq: W-SUN, dtype: int64
2024-01-01    7.5
Freq: MS, dtype: float64
```

**Ek bilgi (resmi dokümantasyon):** Slayt 64–66'daki örnekler görsel olarak verildiği için bu kısım, slayt başlıklarına ve pandas dokümantasyonundaki `to_datetime`, `date_range` ve `resample` kullanımına göre yazıldı. Kaynak: pandas.pydata.org/docs/user_guide/timeseries.html

### Özet ve kaynaklar (Slayt 67–68)

İş akışı: veri alma (`read_csv`) → inceleme (`info`, `describe`) → seçme ve filtreleme (`loc`, `iloc`, maske) → temizleme (`dropna`, `fillna`) → birleştirme (`concat`, `merge`) → analiz (`groupby`, `agg`, `transform`) → yeniden şekillendirme (`melt`, `pivot_table`) → zaman serileri.

Kaynaklar: 10 Minutes to pandas (0.22.0); User Guide, Intro to data structures ve Getting started (pandas 2.3.3), pandas.pydata.org.

## Sık yapılan hatalar

- **Series üzerinde `if` kullanmak:** `if df['Age'] > 30:` → `ValueError: The truth value of a Series is ambiguous`. Maske kullan: `df[df['Age'] > 30]`.

PYTHON

```python
import pandas as pd
df = pd.DataFrame({'Age': [22, 38, 26]})
if df['Age'] > 30:
    print('yaşlı')
```

ÇIKTI

```
ValueError: The truth value of a Series is ambiguous. Use a.empty, a.bool(),
↪ a.item(), a.any() or a.all().
```

- **Filtrelemede `and` / `or` yazmak ya da parantezi unutmak:** `&`, `|`, `~` ve her koşul parantez içinde.
- `loc` **ile** `iloc`**'u karıştırmak:** `iloc` etiket kabul etmez; `loc` dilimi bitişi dahil eder.
- `to_csv`**'de** `index=False` **unutmak:** geri okununca fazladan "Unnamed: 0" sütunu oluşur.
- **Farklı uzunlukta listelerle DataFrame kurmak →** `ValueError: All arrays must be of the same length`.
- **Tam sayı sütununda NaN olunca tipin değişmesine şaşırmak:** float64'e yükselir; gerekirse `'Int64'` kullan.
- `groupby`**'ın hemen sonuç ürettiğini sanmak:** bir toplama/dönüştürme/filtre çağrılana kadar yalnızca bir plandır.

## Kısa tekrar soruları

1 `pd.Series([1, 2], index=['a', 'b']) + pd.Series([10, 20], index=['b', 'c'])` sonucu nedir?

2 `df.shape` ile `df.info()` arasındaki sözdizimi farkı nedir?

3 `x = pd.DataFrame({'v': [5, 6, 7, 8]})` için `x.loc[1:2, 'v'].tolist()` ve `x.iloc[1:2, 0].tolist()` nedir?

4 `df[(df['Age'] > 30) & (df['Sex'] == 'male')]` neden parantez içerir?

5 `dropna(thresh=2)` ne yapar?

6 inner ve outer merge arasındaki fark nedir? `validate` neyi değiştirir?

7 transform ile agg arasındaki fark nedir?

8 pivot_table'da aggfunc verilmezse ne kullanılır?

9 İki Timestamp'in farkının tipi nedir?

## Cevaplar

1 `a` NaN, `b` 12.0, `c` NaN (index'ler hizalanır, birleşimdir).

2 `shape` bir özniteliktir (parantezsiz, tuple döndürür); `info()` bir metottur (parantezle çağrılır, özeti yazdırır).

3 `[6, 7]` (loc'ta bitiş dahil) ve `[6]` (iloc'ta bitiş hariç).

4 `&` karşılaştırma operatörlerinden önce işlendiği için her koşul ayrı ayrı paranteze alınmalıdır.

5 En az 2 eksik olmayan değeri olan satırları tutar, diğerlerini siler.

6 inner yalnızca iki tabloda da olan anahtarları, outer herhangi birinde olan tüm anahtarları tutar. `validate` çıktıyı değiştirmez; anahtar benzersizliği varsayımı bozulursa hata verir.

7 `agg` her grup için tek değer üretir (grup anahtarlarıyla indeksli); `transform` orijinalle aynı şekilde bir sonuç verir, grup değerini her satıra yayar.

8 Ortalama (mean).

9 `Timedelta`.